

Omokai Robotics Take-Home

Challenge 2: Autonomous Navigation

This repository is my submission for Challenge 2 of the Omokai robotics take-home: getting a robot to navigate an environment it doesn't have a fixed map of, by reasoning about what its sensors are telling it rather than following a pre-set list of waypoints.

The robot is a Husky ground vehicle fitted with a single Intel RealSense RGB-D camera. It drives itself from a start marker to a goal marker across a five-lane farm field, working out where the lanes are as it goes and covering them along the way. All of the lane and obstacle detection comes purely from depth. I fit the ground plane with RANSAC and classify everything else by how far it sits above that plane, so there's no colour thresholding involved at all. Navigation itself runs on Nav2, and localization is handled by odometry together with RTAB-Map visual SLAM, which I kept deliberately separate from Nav2's transform tree since its corrections were too jumpy to trust for tight in-lane driving. The full design writeup, including which parameters are hardcoded versus perceived, and the most recent verified run, lives in `WRITEUP.md`.

One thing worth being upfront about: this codebase started life as a personal project I built earlier with a friend, originally for a different company's take-home assignment. I'm reusing it here, which the brief explicitly allows. I've adapted it for Omokai's Challenge 2 and re-verified it end to end. The one piece I've deliberately left out is plant counting, which was the point of the original assignment but has nothing to do with this one. More on that in the footnote at the end.

Packages

Package	Role
<code>farm_description</code>	Husky + RealSense URDF/Xacro, Gazebo plugins (planar-move drive, depth camera)
<code>farm_perception</code>	<code>lane_perception_node</code> : RANSAC ground-plane fit and height classification, producing obstacles and lane centres. This is the piece the challenge is actually graded on.
<code>farm_navigation</code>	<code>marker_reader_node</code> reads the start/goal at runtime; <code>mission_node</code> runs the perception-driven coverage state machine and sends Nav2 goals; also <code>static_map_odom_node</code> and the Nav2 parameter files
<code>farm_bringup</code>	Top-level launch file, the world file, RViz config, <code>video_recorder_node</code> , <code>path_recorder_node</code>

Running it

There are two ways to run this, depending on whether the machine has an NVIDIA GPU. Both need the Docker image built once, then `colcon build` happens inside the container automatically, everything launches, and the deliverables get written out to `./output`. Below are the exact commands for this machine (no GPU), and the equivalent commands for any other machine.

On this machine (no GPU)

```
# one-time: make sure this user can run docker without sudo
sudo usermod -aG docker $USER
newgrp docker
docker info      # should print info, not "permission denied"

# make the scripts executable
chmod +x run.sh run_nogpu.sh launch_gui.sh

# build the image
sudo docker build -t farm_nav:humble -f docker/Dockerfile docker/

# run (no GPU on this machine, so the software-rendering path)
sudo ./run_nogpu.sh

# after the run, output files are root-owned since the container runs as
# root; fix ownership so they can be moved/deleted normally
sudo chown -R $USER:$USER output/
```

On any other machine

```
# one-time: make sure the user can run docker without sudo
sudo usermod -aG docker $USER
newgrp docker
docker info

# make the scripts executable
chmod +x run.sh run_nogpu.sh launch_gui.sh

# build the image
sudo docker build -t farm_nav:humble -f docker/Dockerfile docker/

# if the machine HAS an NVIDIA GPU + drivers + the NVIDIA container toolkit:
sudo ./run.sh
# or, for a live view with Gazebo and RViz open:
sudo ./launch_gui.sh

# if the machine has NO GPU:
sudo ./run_nogpu.sh

# after the run, fix output ownership the same way
sudo chown -R $USER:$USER output/
```

The GPU path (`run.sh` / `launch_gui.sh`) passes rendering through to the container via `--gpus all` and the NVIDIA container runtime. It's the faster and more dependable option since the depth camera renders on the GPU, the simulation stays close to real-time, and there's no risk of Nav2's lifecycle timeouts tripping under load. The no-GPU path (`run_nogpu.sh`) runs entirely headless: `xvfb` gives Gazebo a virtual display inside the container, and depth-camera rendering falls back to software GL (`llvmpipe`, via `LIBGL_ALWAYS_SOFTWARE=1` and `GALLIUM_DRIVER=llvmpipe`). No host X server or `--gpus` flag is needed. It's slower, and the depth image is noisier than it would be with GPU rendering (I go into exactly how that noise showed up in `WRITEUP.md`), but it's fully functional. It's what I used to validate this workspace

start to finish.

Launch arguments

`use_rviz`, `use_slam`, `capture`, `headless`, `spawn_x/spawn_y/spawn_yaw`. The spawn arguments only control where the robot is physically placed at start. The actual navigation goal and start pose are still read from the markers at runtime, not hardcoded.

Two bugs I hit and fixed along the way

The Dockerfile needed `apt-get upgrade -y` before installing the ROS packages. Without upgrading the base `osrf/ros:humble-desktop-full` image’s system packages first, `libdiagnostic_updater` on that image was ABI-mismatched against the version `ros-humble-navigation2` expected. This didn’t fail at install time. It silently broke Nav2’s `lifecycle_manager` at runtime. The symptom was confusing: the robot simply never moved, `mission_node` kept sending the same goal and never got an accepted response, and it retried forever with the reported path length stuck at 0.0. Adding `apt-get upgrade -y` before the package install resolved it.

The video recorder now finalizes itself as soon as the mission reports done. It used to only flush and close the video file inside its ROS shutdown handler. If the container got stopped abruptly right after a mission finished, a hard `docker rm -f` for instance, or the run script’s container being killed before it could exit cleanly, that handler never ran, the MP4’s moov atom was never written, and the resulting file was corrupted (any player, or `ffprobe`, would report “moov atom not found”). It now listens for the `/mission/summary` message and releases the writer the instant that arrives, so the video is valid on disk the moment the mission is reported complete, no matter how the container is torn down afterward.

Key topics

Topic	Contents
<code>/farm/obstacles</code>	Depth-classified obstacle cloud, the only obstacle source Nav2 uses
<code>/farm/lane_center</code> , <code>/lane/markers</code>	Detected corridor centreline, plus RViz markers
<code>/farm/bed_rows</code>	Accumulated perceived bed-row map, used to work out where the next lane centre is
<code>/farm/start_pose</code> , <code>/farm/goal_pose</code>	Marker poses read at runtime, in the map frame
<code>/mission/status</code> , <code>/mission/summary</code>	State-machine status and the final report (lanes covered/skipped, path length)
<code>/plan</code>	Nav2’s planned path

Outputs

`output/run_video.mp4` (the annotated camera feed, with the detected lane region and nav status overlaid), `output/path_overlay.png` (planned versus actual trajectory), and the `/mission/summary` message itself, which also ends up in `output/mission.log` if you redirect the launch output to a file.

Troubleshooting

“permission denied while trying to connect to the Docker daemon socket.” Your user account isn’t in the docker group, or you haven’t logged back in since being added to it.

```
sudo usermod -aG docker $USER
# then log out and back in (or run 'newgrp docker')
```

No GPU available. Use `./run_nogpu.sh` rather than `./run.sh` or `./launch_gui.sh`. The GPU scripts both request `--gpus all`. On a machine with no NVIDIA driver or container toolkit this fails immediately, and even if it didn’t, Nav2’s lifecycle activation can time out under software-rendering load. `run_nogpu.sh` is built specifically for this case and needs no GPU, no NVIDIA container toolkit, and no host X server.

Files in `./output` won’t delete or overwrite. The container runs as root (there’s no `USER` directive in the Dockerfile), and the recorder nodes write directly into the bind-mounted `./output` folder from inside the container, so the video and image end up owned by root on the host. If a later `rm` or overwrite fails with “permission denied,” run:

```
sudo rm -rf output/*
```

before your next run, or just `sudo chown -R $USER:$USER output/`.

Footnote, plant counting, not part of this submission: the project this was adapted from also had a plant-detection and counting pipeline (`farm_perception/plant_counter_node.py`, publishing `/farm/plant_count` and `/farm/plant_markers`), built for the other take-home’s brief, which scored on plant counts. That node is still sitting in `farm_perception`, but it isn’t launched by `bringup.launch.py` and isn’t part of this submission. Omokai’s Challenge 2 is about navigation, not plant counting, so I left it out.